

Zadanie rekrutacyjne AMPERE POINT

Zarys koncepcyjny — prezentacja + własny projekt

Dawid Cekała · wersja 1 · 15 czerwca 2026

Roboczy plan obu części. Cel: pokazać **sposób myślenia** (definicja problemu → założenia → architektura → realny plan → budżet), a nie objętość. Trzymaj luźny ton — to nie obrona pracy.

Część 1 — Prezentacja o własnym produkcie (ok. 10 min)

Rekomendacja: system „follow-me” na ROSbot XL

To Twój najbogatszy projekt pod kątem **wyzwań projektowych i dochodzenia do architektury** — a o to wprost pytają. Masz tu pełny łańcuch inżynierski: percepcja → estymacja stanu → sterowanie → napęd, plus realne kompromisy do opowiedzenia.

Ramuj to uczciwie: platforma (ROSbot XL, Husarion) była gotowa — **Twój jest system**: percepcja, śledzenie celu, regulator i strojenie. To jest „prawie samemu” i tak to przedstaw.

Alternatywy, gdybyś wolał coś czysto „własnego od zera”: projekt **przekładni** albo **PKM** (bardziej mechaniczne, pełniej autorskie, ale uboższa historia architektury sterowania). Rekomenduję ROSbot — bliżej elektroniki/embedded, czyli i bliżej roli w AMPERE POINT.

Struktura 10 minut (z czasami)

1. **Hook + problem (1 min)** — co miało robić i dlaczego to nietrywialne. Jedno zdanie: „robot ma podążać za człowiekiem/markerem w czasie rzeczywistym, nie gubić celu i nie szarpać”.
2. **Założenia i ograniczenia (1 min)** — platforma skid-steer, sensory, budżet obliczeniowy on-board, czas reakcji.
3. **Architektura — serce prezentacji (3-4 min)** — przejdź łańcuch: kamera/ArUco/YOLO → estymacja (Kalman, śledzenie celu) → regulator (MPC/LQR) → kinematyka unicycle/skid-steer → napęd. Pokaż **jeden schemat blokowy** (masz go już z artykułu).
4. **Wyzwania i decyzje (3 min)** — 2-3 konkretne rozwidlenia (patrz niżej). To tutaj pokazujesz myślenie.
5. **Wynik + czego się nauczyłem (1 min)** — co działa, co byś zrobił inaczej, co dalej.

Co podkreślić (wyzwania → decyzje, konkrety)

Wybierz 2-3, nie wszystkie — lepiej głęboko niż płytko:

- **ArUco vs YOLO** — marker (tani, pewny, ale sztuczny) kontra detekcja osoby (ogólniejsza, ale cięższa obliczeniowo i mniej stabilna). Dlaczego wybrałeś to, co wybrałeś — kompromis pewność/koszt CPU.
- **MPC vs LQR** — predykcja i ograniczenia (MPC) kontra prostota i lekkość (LQR). Co wygrało i dlaczego, gdzie był koszt obliczeniowy.
- **Filtr Kalmana / śledzenie celu** — po co estymacja, jak radziłeś sobie z chwilową utratą celu i szumem pomiaru („surowy pomiar szarpał, więc...”).

- **Kinematyka skid-steer** — poślizg kół, dlaczego odometria kłamie i jak to obchodzić.

Ton i 3 pułapki

- Luźno, pierwsza osoba, „pokażę wam jak do tego doszedłem”, nie wykład.
- **Pułapka 1:** za dużo teorii (równania Riccatiego) zamiast decyzji. Trzymaj „problem → opcje → wybór → efekt”.
- **Pułapka 2:** brak konkretnego „co nie działało”. Inżynierowie kochają historię debugowania.
- **Pułapka 3:** przegadanie czasu. Zrób próbę na zegarku, celuj w 9 min.

Mostek do roli (powiedz to wprost na końcu)

„Ten sam sposób myślenia — łańcuch sygnał → estymacja → decyzja → wykonanie, i diagnozowanie, co realnie zawodzi — przenosi się 1:1 na serwis i jakość ładowarek.”
Łączysz robotykę z ich światem.

Część 2 — Własny projekt: Symulator pojazdu / tester EVSE

Urządzenie elektroniczne, które **udaje samochód elektryczny** na stole, żeby testować i diagnozować ładowarki AC bez prawdziwego auta. Trafia w to, co sam zapisałeś o tej pracy („budowanie testerów/symulatorów do szybkiej diagnozy”) i w model firmy (marka serwisująca sprzęt OEM + kontrola jakości u fabryki).

Definicja problemu

Serwis i kontrola jakości ładowarki AC wymagają auta do wpięcia. Ale auto: jest drogie, zajmuje miejsce, daje tylko „grzeczne” odpowiedzi (nie zmusisz go łatwo do zachowań brzegowych) i **nie da się go zautomatyzować** do powtarzalnych testów regresyjnych po zmianie firmware. Efekt: trudno szybko odtworzyć zgłoszenie z terenu i trudno seryjnie sprawdzać partie od fabryki.

Cel

Kompaktowy przyrząd, który emuluje stronę pojazdu w handshake Type 2 / IEC 61851, więc pozwala **bez auta**: przeprowadzić ładowarkę przez stany A→B→C, odczytać oferowany prąd (wypełnienie PWM), opcjonalnie pobrać reprezentacyjny prąd dla weryfikacji stycznego i licznika, **wstrzykiwać usterki** (zła linia CP, brak diody, złe kodowanie PP, nagłe zmiany stanu, wolna reakcja) i wszystko **logować z wynikiem pass/fail**.

Założenia projektowe

- Tylko ładowanie **AC** (Type 2, 1- i 3-fazowe), podstawowa komunikacja **IEC 61851** (PWM na Control Pilot). Bez ISO 15118 / PLC w v1 — świadomie poza zakresem.
- **Bezpieczeństwo najpierw**. Wersja 1 pracuje po stronie niskonapięciowej sygnalizacji (CP/PP) i tylko **izolowanie wykrywa** obecność napięcia na torze mocy. Realny pobór prądu = osobny, poprawnie dobrany moduł obciążenia (v2, z przeglądem bezpieczeństwa).
- Narzędzie dla technika na stanowisku — **uzupełnia**, nie zastępuje certyfikowanych mierników bezpieczeństwa.
- Mikrokontrolerowe, z ekranem i logowaniem (SD/USB/WiFi). Cel budżetu rdzenia:

niskie setki zł.

Architektura — co realnie robi

1. **Emulacja CP (Control Pilot)** — sieć po stronie auta: rezystor 2,74 kΩ + dioda szeregową do PE, oraz dołączany 1,3 kΩ (przełącznik/MOSFET) do przejścia stan B (9 V) → stan C (6 V). Ładowarka „widzi” podłączone, gotowe auto.
2. **Dekoder PWM / odczyt prądu** — pomiar fali CP: amplituda (± 12 V), częstotliwość (1 kHz), wypełnienie → przeliczenie na oferowany prąd ($A = \text{wypełnienie \%} \times 0,6$; $32 \text{ A} \approx 53\%$). MCU mierzy czas stanu wysokiego (input-capture/timer). Weryfikacja, czy ładowarka oferuje właściwy prąd.
3. **Emulacja PP (Proximity Pilot)** — przełączany rezystor (np. 1,5 kΩ / 680 Ω / 220 Ω) udający różną obciążalność kabla; sprawdzasz, czy ładowarka to respektuje.
4. **Weryfikacja toru mocy (rozszerzenie)** — izolowany detektor zamknięcia stycznika (czy załącza, gdy powinna) i opcjonalne, poprawnie dobrane obciążenie rezystancyjne do pobrania zdefiniowanego prądu (test licznika i styków).
5. **Wstrzykiwanie usterek (najwartościowsze)** — celowe stany brzegowe: brak diody (ładowarka ma odmówić), zwarcie CP do PE (stan E), szybkie B→C→B, bardzo wolna reakcja, PP w rozwarciu/zwarcu. Sprawdzasz, czy ładowarka reaguje bezpiecznie (otwiera stycznik, zgłasza błąd). To odtwarza zgłoszenia z terenu.
6. **Automatyzacja + logowanie** — gotowe sekwencje testów uruchamiane automatycznie, wynik z czasem i pass/fail, eksport po USB/WiFi. Umożliwia regresję między wersjami firmware i raport jakości do fabryki.

Co realnie trzeba zrobić (plan fazowy)

- **Faza 0 (1-2 dni):** lektura IEC 61851-1 (CP/PP), spis przypadków testowych.
- **Faza 1 (1-2 weekendy):** front-end analogowy CP na płytce stykowej — obsługa ± 12 V, dzielnik/clamp do 0–3,3 V, przełączanie stanów + dekod PWM. Test najpierw z generatorem udającym ładowarkę, potem z realną ładowarką.
- **Faza 2 (1 weekend):** emulacja PP, ekran OLED/TFT + enkoder, logowanie (SD/serial).
- **Faza 3 (1-2 weekendy):** sekwencje automatyczne + wstrzykiwanie usterek + logika pass/fail.
- **Faza 4 (opcjonalnie, osobno):** moduł obciążenia mocy — stycznik o właściwym prądzie, obciążenie, czujnik prądu, bezpieczniki — z przeglądem bezpieczeństwa.
- **Walidacja:** 2-3 realne jednostki AMPERE POINT + ewentualnie konkurencja.

Wykonalność w pojedynkę — szczerą oceną

Tak, rdzeń jest wykonalny solo. To klasyczny projekt embedded: MCU (ESP32/STM32), front-end op-amp/komparator, kilka przełączników/MOSFET-ów, rezystory precyzyjne, timer/ADC, OLED, SD/WiFi. Główne wyzwanie analogowe to interfejs ± 12 V CP — ale jest dobrze opisany (m.in. open-source **OpenEVSE**), więc nie projektujesz w ciemno.

Ryzyka, które nazywam wprost (to też punktuje): (a) tor mocy dotyka napięcia sieci — dlatego v1 zostaje na niskim napięciu + izolowany pomiar, a obciążenie to v2 z zabezpieczeniami; (b) poprawne wykrycie diody i poziomów CP wymaga starannej kalibracji; (c) to przyrząd diagnostyczny, nie certyfikowany tester bezpieczeństwa — i tak go pozycjonuję.

Budżet estymowany

Element (rdzeń diagnostyczny)	Koszt
Mikrokontroler (ESP32/STM32)	40–80 zł
Front-end CP (op-amp/komparator, rezystory, dioda, clamp)	30–60 zł
Przełączanie rezystorów (przełączniki/MOSFET)	20–40 zł
Emulacja PP (rezystory + przełącznik)	10–20 zł
Wyświetlacz OLED/TFT + enkoder	30–60 zł
Złącze Type 2 (gniazdo/wtyk lub tani kabel do rozbiórki)	100–250 zł
Płytki/obudowa/okablowanie/zasilacz ± 12 V/logowanie	80–150 zł
Razem rdzeń	ok. 400–700 zł
Opcjonalny stopień mocy (stycznik + obciążenie + czujnik + bezpieczniki)	+300–700 zł

Realnie: **ok. 500 zł za sam rdzeń diagnostyczny**, do ok. 1 200–1 400 zł z prostym stopniem obciążenia. Daleko od „laboratorium za milion”, a na wyjściu masz **reżywalny przyrząd**, nie jednorazowe potwierdzenie fizyki.

Narzędzia

- **Prototypowanie:** ESP32/STM32 + Arduino/PlatformIO/STM32Cube, płytka stykowa, multimetr, oscyloskop (kluczowy do podglądu CP ± 12 V/PWM — wystarczy tani USB-scope), generator (albo drugi MCU udający ładowarkę), lutownica.
- **Projekt:** KiCad (schemat/PCB), Wokwi/Falstad (symulacja obwodu).
- **Referencja:** open-source **OpenEVSE** (interfejs CP), norma IEC 61851-1.
- **Dokumentacja/prezentacja:** Twój zwykły toolchain (LaTeX/Markdown).

Dlaczego to wartościowe (nie tylko fizyka)

To narzędzie, którego dział serwisu/QA używałby codziennie: odtwarzanie usterek z terenu, regresja firmware, walidacja partii z fabryki — dokładnie obieg marki-OEM, o którym rozmawialiśmy. Zamienia wiedzę diagnostyczną w **powtarzalny, automatyzowalny przyrząd**. To przeciwieństwo jednorazowego demo z podręcznika.

Checklista do poniedziałku

- ☐ Część 1: wybierz projekt (rekom. ROSbot), wytnij schemat blokowy, wypisz 2–3 wyzwania-decyzje, zrób próbę na zegarku (do 9 min).
- ☐ Część 2: 1 slajd problemu, 1 schemat architektury (CP/PP/MCU), 1 tabela budżetu, 1 zdanie o bezpieczeństwie/zakresie v1.
- ☐ Spięcie: jedno zdanie łączące robotykę → serwis/jakość ładowarek.
- ☐ Nie przepalaj czasu — to ma pokazać myślenie, nie być produktem końcowym.